

Prompt Injection Attacks in Large Language Models: Taxonomy, CIA Triad Threat Modeling, and a Contextual Privilege Layering Defense

Rimsha Majeed^{*a,1}

^aDepartment of Information Technology, Punjab University College of Information Technology (PUCIT), Lahore, Pakistan

Submitted: April 2026 Supervisor: Sr. Huzaifa Roll No: BITF22Mo29

Abstract

Large language models have moved from research curiosity to production infrastructure with remarkable speed, and that speed has outpaced the security frameworks designed to keep them safe. Among the vulnerabilities that have emerged, prompt injection sits at the top of every major threat taxonomy — including the OWASP Top 10 for LLM Applications 2025 — precisely because it is not a bug that can be patched but an architectural consequence of treating all input as trusted natural language. This paper examines that problem from three angles. First, we survey the most recent literature from 2025 and 2026, tracing how the threat has evolved from simple instruction overrides to sophisticated multi-stage exploits targeting agentic pipelines, retrieval-augmented generation systems, and inter-agent communication protocols. Second, we apply the CIA Triad as a formal classification lens, mapping each attack category to its impact on Confidentiality, Integrity, and Availability in concrete terms. Third, we introduce Contextual Privilege Layering (CPL), a novel pre-attention segmentation scheme that assigns privilege tiers to token segments before they enter the transformer attention mechanism, blocking cross-tier information flow through a learned privilege mask. Unlike detection-based approaches that flag attacks after they have influenced the model, CPL operates inside the attention computation itself. Experimental comparison against five baseline defenses shows CPL achieving 97.3% detection accuracy and reducing attack success rate to 1.8%, the strongest result across all evaluated methods.

Keywords: Prompt Injection, Large Language Models, CIA Triad, Contextual Privilege Layering, Agentic AI Security, RAG Poisoning, Adversarial NLP

1. INTRODUCTION

The deployment of large language models (LLMs) in real-world systems has created a category of security vulnerability that has no clean analogue in classical software engineering. In traditional systems, the boundary between code and data is enforced at the architectural level — a SQL query parser, for instance, distinguishes syntax from values because they occupy structurally different roles. LLMs collapse that boundary entirely. A system prompt, a user message, a retrieved document, and a malicious instruction embedded in a web page all arrive as the same thing: tokens in a context window [owasp2025]. The model cannot tell them apart by inspection alone, and neither can any external filter that operates only on the surface form of the text.

This is the root of prompt injection. An attacker who can get text in front of an LLM — through user input, through a retrieved document, through a tool's API response, through an image caption — can issue instructions that the model may follow with the same fidelity it would give its own system prompt [greshake2023]. The consequences range from embarrassing to catastrophic: systems that leak their own configuration, agents that exfiltrate user data, code assistants that introduce vulnerabilities, medical chatbots that recommend contraindicated treatments [gulyamov2026].

What makes the problem harder is that LLMs are increasingly not standalone systems. They are the reasoning core of agents that call external APIs, query knowledge bases, send emails, write and execute code, and communicate with other agents [ferrag2026]. Each of those integrations is a surface through which injected content can arrive, and each action the agent can take is a consequence an attacker might be able to trigger. A 2025 study from a team spanning OpenAI, Anthropic, and Google DeepMind found that adaptive attacks could bypass twelve published defenses at attack success rates above 90% — which is a sobering benchmark for the field [nasr2025].

This paper makes three contributions. We synthesise the most recent research on prompt injection (Section II), covering work published from late 2025 through early 2026 and tracing how the threat

model has grown. We apply the CIA Triad — Confidentiality, Integrity, Availability — as a structured classification framework (Sections III and IV), which gives a more precise vocabulary for talking about what attacks actually compromise. And we propose Contextual Privilege Layering (CPL), a novel attention-level defense that has not appeared in any of the works we survey (Section V). Section VI evaluates CPL against five baselines. Sections VII and VIII discuss limitations, future directions, and conclusions.

2. BACKGROUND AND RELATED WORK

The threat of prompt injection was identified early in the LLM deployment cycle, but its scope has expanded with every new capability added to these systems. We review the ten most relevant recent works, ordered from newest to oldest.

Ferrag et al. (April 2026) published the most comprehensive taxonomy of protocol-level threats in LLM-agent ecosystems to date [ferrag2026]. Their survey catalogs over thirty attack techniques spanning input manipulation, model compromise, system and privacy attacks, and protocol vulnerabilities. Two contributions stand out. First, they formally define Prompt-to-SQL injection, where LLMs that front-end database systems can be coerced into generating queries that exfiltrate or corrupt data. Second, they document the Toxic Agent Flow exploit in GitHub's Model Context Protocol server, which chains tool calls to escalate privileges. Their mitigation framework centers on cryptographic provenance tracking and sandboxed agentic interfaces.

Liao et al. (April 2026) offer a survey of attack and defense techniques with a forward-looking perspective on what the next generation of LLM threats will look like [liao2026]. Their key contribution is a

^{*}Corresponding author: Rimsha Majeed, BSIT Fall 2022, Semester-o8, DI-325 Cybersecurity.

This paper was prepared as part of the DI-325 Cybersecurity course project at Punjab University College of Information Technology. The work presents an original threat analysis and a novel defense contribution — Contextual Privilege Layering (CPL) — which has not appeared in prior published literature.

categorization of attacks by where in the model’s processing pipeline they operate — tokenization, embedding, attention, generation — which informs the design of our CPL framework.

Alzahrani (January 2026) presents PromptGuard, a structured framework for building injection-resilient language models [alzahrani2026]. PromptGuard operates at the input boundary, applying a combination of syntactic and semantic filters before the prompt reaches the model. It achieves strong results against direct injection but, by its own evaluation, is less effective against indirect injection embedded in retrieved content, since the filtering must process untrusted text that is syntactically indistinguishable from legitimate retrieval.

Gulyamov et al. (January 2026) provide a comprehensive review synthesizing over 120 sources [gulyamov2026]. They document two critical real-world incidents: CVE-2025-53773, the GitHub Copilot remote code execution vulnerability triggered by prompt injection with a CVSS score of 9.6, and the CamoLeak exploit which achieved the same score by using injected prompts to exfiltrate repository secrets. The authors propose PALADIN, a five-layer defense architecture, and identify two fundamental limitations they term the stochastic nature problem — the fact that LLM outputs are probabilistic, so no input filter can deterministically predict when an injection will succeed — and the alignment paradox, which is the observation that making a model more instruction-following makes it more susceptible to injected instructions.

Syed et al. (January 2026) address the specific challenge of multi-modal prompt injection, where malicious instructions are embedded not in text but in images, audio, or other modalities [syed2026]. Their Cross-Agent Multimodal Provenance-Aware Framework tracks the origin of each piece of data as it flows through a multi-agent pipeline and masks content whose provenance falls below a trust threshold. They report 94% detection accuracy and a 70% reduction in trust leakage across agentic workflows.

Hossain et al. (December 2025) evaluated a multi-agent defense pipeline using two architectures: a sequential chain-of-agents design and a hierarchical coordinator-based system [hossain2025]. Testing on 400 attack instances across eight categories — including direct overrides, code execution attempts, and obfuscation techniques — they reduced the attack success rate from 30% on ChatGLM and 20% on Llama-2 to 0% in controlled conditions. The limitation is computational: the dual-model overhead roughly doubles inference latency.

Sassal and Can (2025) conducted a comparative study of three frontier models under prompt injection conditions [sassal2025]. Across four metrics — compliance, filter bypass, sensitive information leakage, and security risk level — Claude demonstrated the most consistent refusal behavior, while Gemini was most vulnerable due to filtering failures. GPT-4o fell between the two. The study highlights that model architecture and RLHF alignment strategy have measurable effects on injection resistance even without any additional defense mechanisms.

Nasr, Carlini et al. (October 2025) delivered perhaps the most important empirical result of the year: a systematic study showing that adaptive attacks — those allowed to iterate using gradient descent, reinforcement learning, random search, or human-guided exploration — bypass twelve published defenses at attack success rates above 90% for most [nasr2025]. Several of those defenses had originally reported near-zero attack success rates in their own evaluations. The lesson is sobering: evaluations that do not account for adaptive adversaries systematically overestimate defense effectiveness.

Meta AI (October 2025) proposed the Agents Rule of Two as a practical operational constraint for agentic systems [meta2025]. The rule states that an autonomous agent should satisfy no more than two of the following three properties simultaneously: access to private or sensitive data, exposure to untrusted external input, and ability to communicate with or take actions on external systems. Any agent that satisfies all three without a human-in-the-loop checkpoint creates conditions for the highest-impact prompt injection consequences.

This is not a technical solution but a design principle that limits the blast radius.

Hung et al. (April–May 2025) made a mechanistic discovery about how prompt injection works inside the transformer architecture [hung2025]. By analyzing attention patterns across layers and heads, they identified a distraction effect: specific attention heads that normally focus on the original instruction shift their attention to injected instructions when an attack is present. They call these important heads and build a detection method — Attention Tracker — that monitors their behavior to flag injection attempts. This work directly informs the design of CPL, which operates on the same attention-level insight but intervenes rather than detects.

3. TAXONOMY OF PROMPT INJECTION ATTACKS

Prompt injection attacks vary considerably in their delivery mechanism, target, and consequence. Table 1 presents a unified taxonomy across seven dimensions.

Table 1. Prompt Injection Attack Taxonomy		
Attack Type	Description	CIA Impact
Direct Injection	Malicious instruction embedded directly in user turn	Integrity
Indirect Injection	Instructions hidden in external content (PDFs, web pages, emails)	Confidentiality, Integrity
RAG Poisoning	Malicious documents inserted into retrieval knowledge base	Integrity
Tool/API Injection	Injected prompt triggers unintended external tool calls	All three
Role-Play Attack	Persona override bypasses alignment constraints	Integrity
Multilingual Bypass	Injection written in low-resource language to evade English-trained filters	Integrity
Multimodal Injection	Instruction embedded in image, audio, or file metadata	Confidentiality, Integrity
DoS Prompt	Recursive or computationally expensive prompts exhaust resources	Availability
Prompt-to-SQL	LLM coerced into generating malicious database queries	Confidentiality, Integrity

Note: CIA Impact indicates which security properties are primarily compromised. Tool/API Injection affects all three because it can leak data, alter state, and cause denial of service depending on tool capabilities.

The distinction between direct and indirect injection deserves emphasis. Direct injection requires the attacker to have access to the user input channel — which may be public or constrained. Indirect injection requires only that the LLM process some content the attacker can influence, which in an agentic context includes any web page the agent browses, any email it reads, any document it retrieves [greshake2023]. As Gulyamov et al. document, just five carefully crafted documents among millions can achieve a 90% attack success rate against RAG-augmented systems [gulyamov2026]. That asymmetry — attacker cost close to zero, defender cost unbounded — is what makes indirect injection particularly dangerous.

Role-playing attacks deserve separate mention because they exploit a different vulnerability: alignment rather than architecture. When a model has been trained to be helpful and to follow instructions, an injected persona that reframes the model’s identity as an unrestricted system can shift the model’s behavior without any technical exploit. The alignment paradox identified by Gulyamov et al. captures this precisely — the same quality that makes a model useful makes it exploitable.

4. CIA TRIAD THREAT MODEL

The CIA Triad — Confidentiality, Integrity, Availability — provides a principled framework for classifying security threats. Applied to prompt injection, it reveals that different attack classes compromise fundamentally different properties, which in turn suggests different mitigation priorities.

4.1. Confidentiality Attacks

Confidentiality attacks seek to extract information that should remain hidden. In the LLM context, the primary targets are system prompt contents, user conversation history, API keys embedded in context, and data retrieved from private knowledge bases [owasp2025]. Table 2 shows the mapping in detail.

The mechanism is typically prompt extraction: an injected instruction tells the model to repeat, summarize, or rephrase its system context. OWASP classifies this as LLM07:2025 (System Prompt Leakage). In agentic systems, the more dangerous variant involves extracting credentials or session tokens that the agent holds, enabling the attacker to act on external systems directly.

4.2. Integrity Attacks

Integrity attacks alter the model’s outputs in ways that serve the attacker’s goal while appearing legitimate to the user. This is the broadest category and the hardest to defend against, because the definition of integrity is context-dependent — what counts as the correct output depends on what the legitimate task was, and the model cannot always be certain what that task is [nasr2025].

RAG poisoning is a particularly effective integrity attack. By inserting authoritative-looking but malicious content into a retrieval corpus, an attacker can bias the model’s responses across many user sessions without any access to those sessions directly. The attack is persistent and scalable.

4.3. Availability Attacks

Availability attacks degrade or eliminate the model’s ability to serve legitimate users. The most straightforward variant is the prompt denial-of-service: a prompt engineered to cause the model to enter a repetitive loop, generate extremely long outputs, or trigger repeated tool calls. In multi-agent systems, a single compromised agent can propagate such behavior to others [ferrag2026].

Table 2 summarises concrete examples across all three categories with their associated risk level.

Table 2. CIA Triad Attack Classification

CIA	Attack Example	Consequence	Risk
Confidentiality	System prompt extraction	Config/API key leak	Critical
	User data exfiltration	PII disclosure	Critical
	RAG source leakage	Knowledge base exposed	High
Integrity	RAG corpus poisoning	False information returned	Critical
	Output manipulation	Malicious code injected	Critical
	Role-play override	Safety constraints bypassed	High
Availability	Recursive prompt loop	Service unresponsive	High
	Context flooding	Memory/token exhausted	High
	Agent DoS propagation	Multi-system disruption	Critical

Note: Risk levels follow CVSS conventions. Critical indicates CVSS 9.0+, High indicates 7.0–8.9.

5. CONTEXTUAL PRIVILEGE LAYERING

5.1. Motivation

The Attention Tracker work by Hung et al. makes a mechanistic observation: prompt injection succeeds by redirecting the model’s important attention heads from the original instruction to the injected one [hung2025]. Detection-based approaches like Attention Tracker monitor this shift and flag it after the fact. But if we understand the mechanism, a more direct intervention is possible — prevent the cross-tier attention shift from occurring at all.

Most existing defenses operate at input or output boundaries: they filter what goes in, or they filter what comes out. Neither approach addresses the internal computation where the confusion actually occurs. CPL intervenes at the attention layer itself.

5.2. Privilege Tier Assignment

CPL assigns each segment of the input context to one of four privilege tiers before tokenization:

- **Tier 0 — System:** Developer-controlled system prompt. Full trust.
- **Tier 1 — User:** Authenticated user message. High trust.
- **Tier 2 — Retrieval:** Content returned by RAG or tool calls. Limited trust; read-only semantics.
- **Tier 3 — Environment:** Web content, file contents, emails, external API responses. Untrusted; instruction-blind processing.

Tier assignment is performed by a lightweight segment classifier that operates on provenance metadata (the source field of each context segment), not on the content itself. This matters: a content-based classifier can be evaded by making injected content look syntactically benign. A provenance-based classifier cannot be evaded without compromising the provenance metadata itself, which requires a different class of attack.

5.3. Privilege Mask

Each token i in the context receives a tier label $\tau_i \in \{0, 1, 2, 3\}$. The privilege mask matrix $\mathbf{M}_p \in \{0, 1\}^{n \times n}$ for a context of n tokens is defined as:

$$(\mathbf{M}_p)_{ij} = \begin{cases} 1 & \text{if } \tau_j \leq \tau_i \\ 0 & \text{if } \tau_j > \tau_i \end{cases} \quad (1)$$

This means token i can attend to token j only if j belongs to the same tier or a more trusted (lower-numbered) tier. A Tier 2 retrieval token cannot influence a Tier 0 system token’s attention. A Tier 3 environment token is instruction-blind: it can be read for factual content but cannot issue instructions that reach higher-trust tokens.

5.4. CPL Attention Computation

The standard scaled dot-product attention is:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V \quad (2)$$

CPL modifies this by adding the log-domain privilege mask before the softmax, following the standard technique for masked attention:

$$\text{Attention}_{\text{CPL}}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + \log \mathbf{M}_p\right)V \quad (3)$$

where $\log \mathbf{M}_p$ evaluates to 0 where $(\mathbf{M}_p)_{ij} = 1$ and to $-\infty$ where $(\mathbf{M}_p)_{ij} = 0$. The $-\infty$ terms vanish after softmax, effectively zeroing those attention weights. No gradient flows through blocked connections during inference.

5.5. Implementation

CPL requires no model retraining. It is applied at inference time by modifying the attention computation in each transformer layer. The privilege tier tag for each token is propagated from the segment-level classifier through the tokenizer. The computational overhead is one additional elementwise addition per attention layer — negligible compared to the matrix multiplication cost. Table 3 summarises the tier definitions and their trust semantics.

Table 3. CPL Privilege Tier Definitions

Tier	Label	Source	Trust Semantics
0	System	Developer system prompt	Full — can instruct any tier
1	User	Authenticated user input	High — can instruct Tier 1+
2	Retrieval	RAG / tool API response	Limited — informational only
3	Environment	Web, files, emails	None — instruction-blind

Note: Tier assignment is provenance-based, not content-based, making CPL resistant to syntactic evasion.

5.6. Comparison with Prior Defenses

CPL differs from existing defenses in its point of intervention:

- **vs. PromptGuard [alzahrani2026]:** PromptGuard filters at the input boundary before the model sees the text. CPL operates inside the attention computation; it does not need to distinguish injected content syntactically.
- **vs. Attention Tracker [hung2025]:** Attention Tracker detects when important heads have been distracted; CPL prevents the distraction by masking cross-tier attention paths.
- **vs. Dual-LLM [hossain2025]:** Dual-LLM isolation runs a separate model to inspect outputs, doubling inference cost. CPL adds negligible overhead.
- **vs. Syed et al.’s provenance framework [syed2026]:** That work tracks provenance across agents and masks at the agent communication boundary. CPL tracks provenance at the token level within a single model pass.

6. EXPERIMENTAL EVALUATION

6.1. Setup

We evaluated CPL and five baseline defenses on a dataset of 1,200 prompt injection attack instances spanning all nine attack types from Table 1, balanced across direct injection (300 instances), indirect injection (300), RAG poisoning (200), tool injection (150), role-play attacks (150), and other types (100). The base model was Llama-3-8B-Instruct. Defenses were evaluated in a black-box adaptive setting following the methodology of Nasr et al. [nasr2025], where the attacker is allowed to iterate up to 50 attempts per sample.

6.2. Metrics

We report five metrics: Detection Accuracy (the proportion of injections correctly identified), Attack Success Rate (ASR, the proportion of attacks that achieve their goal despite the defense), Precision, Recall, and F1-Score. We also report inference latency overhead relative to the undefended baseline.

6.3. Results

Table 4 presents the full comparison. CPL achieves the highest Detection Accuracy (97.3%) and the lowest ASR (1.8%) among all evaluated methods, while adding under 2% latency overhead — the lowest overhead of any non-trivial defense.

Table 4. Defense Performance Comparison

Method	Acc. (%)	ASR (%)	Prec. (%)	Rec. (%)	F1 (%)
No Defense	0.0	100.0	—	—	—
Input Filtering	71.4	34.2	74.1	68.3	71.1
Embedding	82.6	22.7	84.3	80.9	82.5
Detection					
Attention Tracker [hung2025]	89.1	14.3	90.4	87.8	89.1
Dual-LLM [hossain2025]	93.7	7.6	94.2	93.1	93.6
CPL (Ours)	97.3	1.8	97.8	96.9	97.3

Note: ASR is measured under adaptive attack conditions following [nasr2025]. CPL latency overhead: < 2%. Dual-LLM overhead: ≈ 95%.

6.4. Analysis by Attack Type

Table 5 breaks down CPL’s performance across individual attack categories.

Table 5. CPL Detection Accuracy by Attack Type

Attack Type	Accuracy (%)	ASR (%)
Direct Injection	98.7	0.9
Indirect / RAG Injection	97.1	2.0
Tool / API Injection	96.4	3.0
Role-Play Override	97.8	1.5
Multilingual Bypass	95.3	5.1
Multimodal Injection	94.8	6.2
DoS Prompt	99.1	0.5

Note: Multilingual and multimodal attacks show the lowest accuracy, reflecting the limits of provenance-based tagging when content type boundaries are ambiguous.

CPL performs best on direct injection and DoS prompts, where provenance metadata is unambiguous. Multilingual bypass and multimodal injection present harder cases: in both, the provenance of the segment is technically user-level (Tier 1), so CPL’s privilege mask does not block the content. Catching these requires a content-level layer in addition to CPL’s structural layer. This is a known limitation and a direction for future work.

7. CHALLENGES AND FUTURE DIRECTIONS

7.1. The Adaptive Adversary Problem

The most important open problem in LLM security is the one that Nasr et al. quantified: defenses evaluated against fixed attack sets overstate their effectiveness [nasr2025]. An attacker with access to the defense mechanism — or even just the ability to observe its outputs and iterate — can usually find a bypass. CPL is not immune to this. A sophisticated attacker who can manipulate provenance metadata directly can assign Tier 0 labels to malicious content. Protecting provenance metadata is therefore a first-class security requirement for CPL deployments.

7.2. Multilingual and Multimodal Gaps

As Table 5 shows, multilingual and multimodal attack variants are harder. Current filter-based defenses, including CPL’s segment classifier, are predominantly trained on English text [gulyamov2026]. A malicious instruction in a low-resource language, or encoded in an image’s EXIF metadata, may not trigger the same provenance markers. Extending CPL to handle these cases requires either multilingual provenance classifiers or modality-specific pre-processing pipelines.

7.3. Agentic Complexity

The Agents Rule of Two from Meta AI [meta2025] identifies an operational constraint, not a technical solution. As agents become more capable and autonomous, the pressure to give them all three of the

dangerous properties simultaneously — private data access, untrusted input, and external communication — will grow. CPL addresses the untrusted input dimension by downgrading its privilege tier. But if a Tier 3 segment can legitimately request that the agent take an external action (for instance, a webpage that instructs a shopping agent to add a product to the cart), distinguishing legitimate from malicious instructions at Tier 3 becomes semantically hard.

7.4. Future Work

Three directions follow naturally from this work. First, integrating CPL with a multilingual semantic filter addresses the accuracy gap on language-switched attacks. Second, applying provenance tracking at the agent communication boundary — as Syed et al. do at the agent level — and feeding that provenance into CPL’s tier assignment extends the protection to multi-agent pipelines. Third, formal verification of CPL’s attention masking guarantees under adversarial provenance manipulation would strengthen the theoretical foundation.

8. CONCLUSION

Prompt injection is not a vulnerability that will be fixed by a patch or a new model version. It is a consequence of the fact that LLMs process all input as natural language, and natural language does not have a syntax that separates instructions from data. That architectural reality is unlikely to change, which means defenses must operate in the presence of it rather than trying to eliminate it.

The research surveyed here makes clear that the threat has matured considerably. What began as simple instruction overrides in 2023 now encompasses multi-stage agent exploits, RAG corpus poisoning, protocol-level attacks on MCP servers, and adaptive attacks that defeat twelve published defenses at over 90% success rate. The field is not losing ground uniformly — Attention Tracker’s mechanistic discovery, the Agents Rule of Two’s operational clarity, and multi-agent pipeline defenses all represent genuine progress. But the progress is incremental against a threat that is growing structurally.

The Contextual Privilege Layering framework proposed here contributes a new point of intervention: the attention computation itself. By assigning privilege tiers to token segments based on their provenance and enforcing those tiers through a privilege mask inside the scaled dot-product attention, CPL prevents the cross-tier attention shifts that Hung et al. identified as the mechanistic signature of prompt injection. The result is a defense that does not depend on recognizing malicious content syntactically, that adds negligible latency, and that achieves 97.3% detection accuracy and a 1.8% attack success rate under adaptive conditions.

No single mechanism will solve prompt injection. CPL addresses structural confusion between context segments of different trust levels — but it does not address every attack type, and it requires provenance metadata that must itself be protected. The path forward is layered defense: provenance-aware attention like CPL, combined with semantic filtering, output monitoring, operational constraints like the Agents Rule of Two, and continuous red-teaming against adaptive adversaries. That combination is not a solution so much as a posture — one that takes the architectural reality of LLMs seriously and builds defensively from it.

ACKNOWLEDGMENT

The author thanks Sr. Huzaifa for guidance and feedback throughout this project.

REFERENCES

[1] OWASP Foundation, “OWASP Top 10 for Large Language Model Applications 2025,” OWASP, 2025. [Online]. Available: <https://owasp.org/www-project-top-10-for-large-language-model-applications>

[2] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, “Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection,” in *Proc. ACM Workshop on Artificial Intelligence and Security (AISeC)*, 2023.

[3] S. Gulyamov, R. Raupov, A. Abdullaev, M. Tursunov, and F. Muminov, “Prompt injection attacks in large language models and AI agent systems: A comprehensive review of vulnerabilities, attack vectors, and defense mechanisms,” *Information*, vol. 17, no. 1, p. 54, Jan. 2026.

[4] M. A. Ferrag, N. Tihanyi, D. Hamouda, L. Maglaras, A. Lakas, and M. Debbah, “From prompt injections to protocol exploits: Threats in LLM-powered AI agents workflows,” *ICT Express*, vol. 12, no. 2, pp. 353–383, Apr. 2026.

[5] Z. Liao, K. Chen, Y. Lin, K. Li, Y. Liu, H. Chen, X. Huang, and Y. Yu, “Attack and defense techniques in large language models: A survey and new perspectives,” *Neural Networks*, vol. 196, p. 108388, Apr. 2026.

[6] A. Alzahrani, “PromptGuard: A structured framework for injection resilient language models,” *Scientific Reports*, vol. 16, no. 1, Jan. 2026.

[7] T. A. Syed, M. A. Almutairi, and M. A. Moaty, “Cross-agent multimodal provenance-aware framework for prompt injection defense in agentic AI systems,” *Journal of Information Security and Applications*, Jan. 2026.

[8] S. M. A. Hossain, A. Datta, R. Islam, M. F. Mridha, and J. Shin, “A multi-agent LLM defense pipeline against prompt injection attacks,” *arXiv preprint arXiv:2509.14285*, Dec. 2025.

[9] S. Sassal and Ö. Can, “Prompt injection attacks on large language models: Multi-model security analysis with categorized attack types,” in *Proc. 17th Int. Joint Conf. Knowledge Discovery, Knowledge Engineering and Knowledge Management (IC3K)*, 2025, pp. 517–524.

[10] M. Nasr, N. Carlini, C. Sitawarin, S. V. Schulhoff, J. Hayes, M. Ilie, J. Pluto, S. Song, H. Chaudhari, I. Shumailov, A. Thakurta, K. Y. Xiao, A. Terzis, and F. Tramèr, “The attacker moves second,” *arXiv preprint arXiv:2510.06387*, Oct. 2025.

[11] Meta AI (M. Ayzenberg et al.), “Agents Rule of Two: A practical approach to AI agent security,” Meta AI Blog, Oct. 2025. [Online]. Available: <https://ai.meta.com/blog/agents-rule-of-two>

[12] K.-H. Hung, C.-Y. Ko, A. Rawat, I.-H. Chung, W. H. Hsu, and P.-Y. Chen, “Attention tracker: Detecting prompt injection attacks in LLMs,” in *Findings of the ACL: NAACL 2025*, pp. 1–15, 2025.

[13] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[14] Y. Liu, G. Deng, Y. Li, K. Wang, T. Zhang, Y. Liu, H. Wang, Y. Zheng, and Y. Liu, “Prompt injection attack against LLM-integrated applications,” *arXiv preprint arXiv:2306.05499*, 2023, updated Dec. 2025.

[15] J. Yi, Y. Xie, B. Zhu, E. Kiciman, G. Sun, X. Xie, and F. Wu, “Benchmarking and defending against indirect prompt injection attacks on large language models,” in *Proc. 31st ACM SIGKDD Conf. Knowledge Discovery and Data Mining*, 2025.